

Redes de Ordenadores

GII – IYA022

Protocolos de Transporte

Mariano Benito

- La capa de transporte
- TCP
- UDP

- Conocer los principales protocolos del nivel de transporte.
- Comprender cómo se establecen y finalizan las comunicaciones TCP.
- Diferenciar las comunicaciones orientadas a conexión y no orientadas a conexión.

Aplicación

Transporte

Red

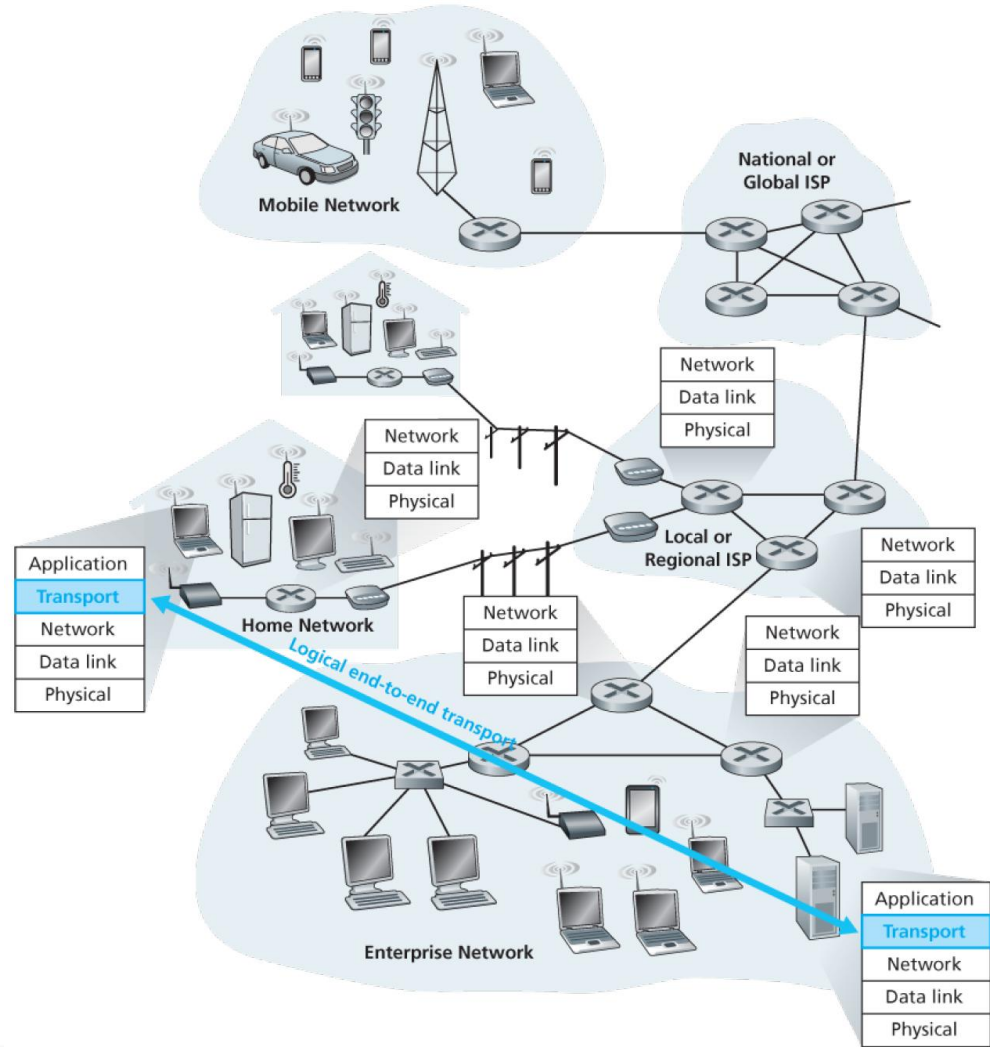
Enlace

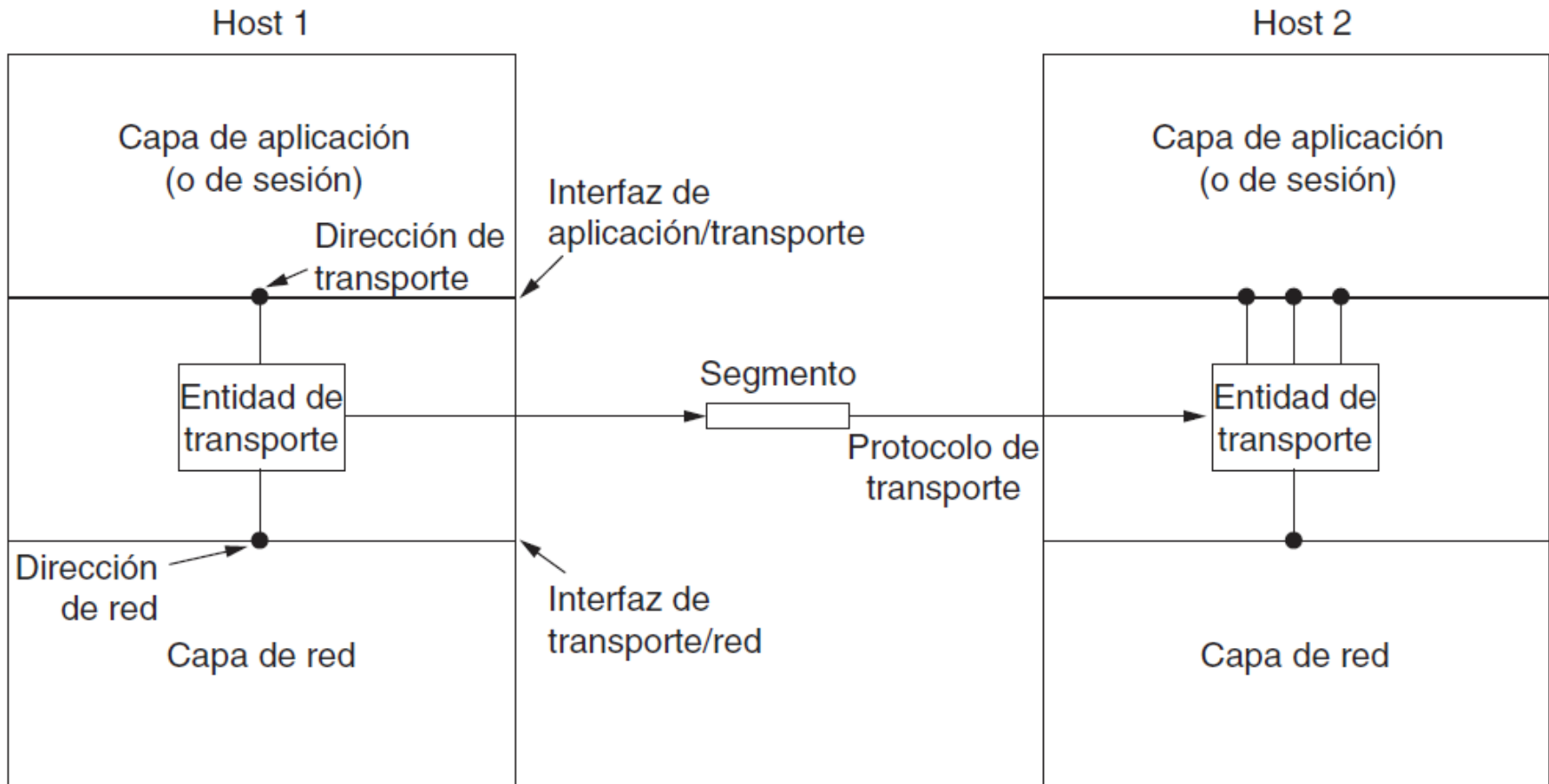
Física

- Un protocolo de transporte **proporciona un servicio lógico de transferencia de datos** entre dos equipos (**extremo a extremo**), empleando los servicios de la capa inferior (red, interred, internet).
- Se encarga de dos aspectos generales:
 - Proporciona una **interfaz con las aplicaciones**, presentando los puntos de acceso al servicio → son el direccionamiento de las aplicaciones.
 - **Implementa tareas comunes a múltiples aplicaciones**: formato de datos, delimitación de mensajes, control de flujo y errores, control de congestión, establecimiento de sesión, multiplexación de flujos, ...
- Su PDU es el **segmento**: encapsula los datos de la capa de aplicación junto con su cabecera.

- La **fiabilidad** extremo a extremo **implica detectar pérdidas y duplicados, recuperar el orden original y confirmar la recepción** → esto introduce **acuses de recibo, temporizadores** y posibles **retransmisiones** que añaden retardo y variabilidad.
- No todas las aplicaciones toleran ese coste, por ello, **coexisten dos protocolos representativos**:
 - **UDP**: servicio no orientado a conexión, rápido, eficiente y sin garantías.
 - **TCP**: servicio orientado a conexión con entrega fiable y ordenada, con control de flujo y de congestión.

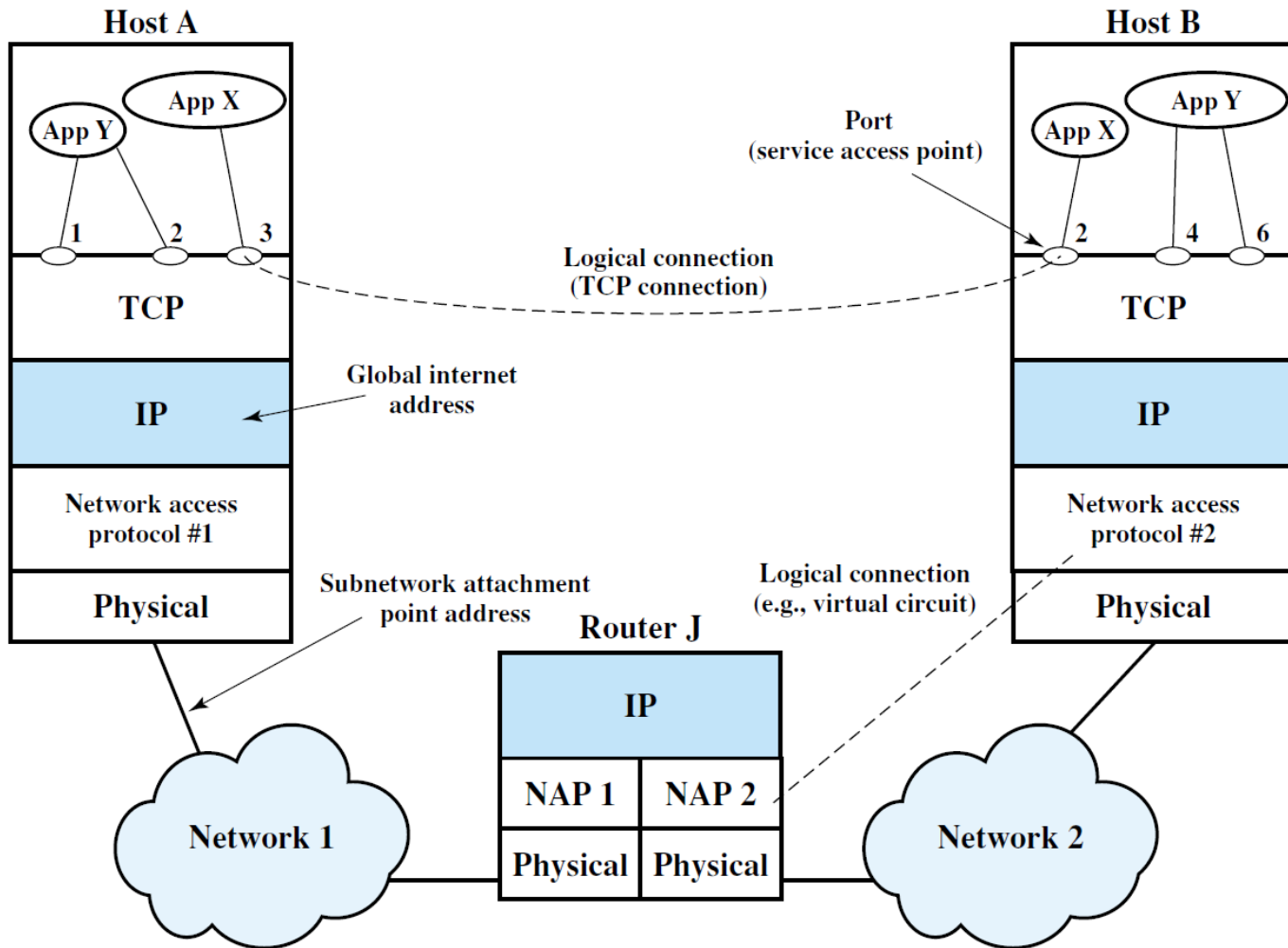
¿Dónde se implementan los protocolos de transporte?





- **Funciones frecuentes del nivel de transporte:**
 1. **Direccionamiento y multiplexación** de aplicaciones: asigna a cada aplicación un punto de acceso al servicio por el que se comunica:
 - **TCP/IP:** puerto TCP o UDP.
 - OSI: TSAP (*Transport Service Access Point*).
 2. **Control de flujo:** el transmisor debe moderar el envío de datos si es capaz de transmitir (y la red es capaz de cursar el tráfico) más rápido que lo que el receptor es capaz de recibir.
 3. **Control de errores:** deben detectarse los errores de transmisión (y pérdidas de datos, por descarte de tramas en niveles inferiores) y solventarse:
 - Usando retransmisiones o envío redundante y corrección en el receptor.
 - Aunque se controlen los errores a nivel de enlace, el nivel de red (en modo datagrama) podría perder tramas.
 4. **Control de congestión:** el transmisor debe moderar el envío de datos si es capaz de transmitir más rápido de lo que la red es capaz de cursar.
 5. **Entrega ordenada:** la red puede entregar datos en desorden, entonces, el protocolo de transporte debe recuperar la secuencia original sin duplicados.
 6. **Establecimiento y liberación de conexiones:** permite iniciar y finalizar una conexión, estableciendo parámetros y reservando recursos.

No todos los protocolos de transporte son iguales ni tienen las mismas funciones.



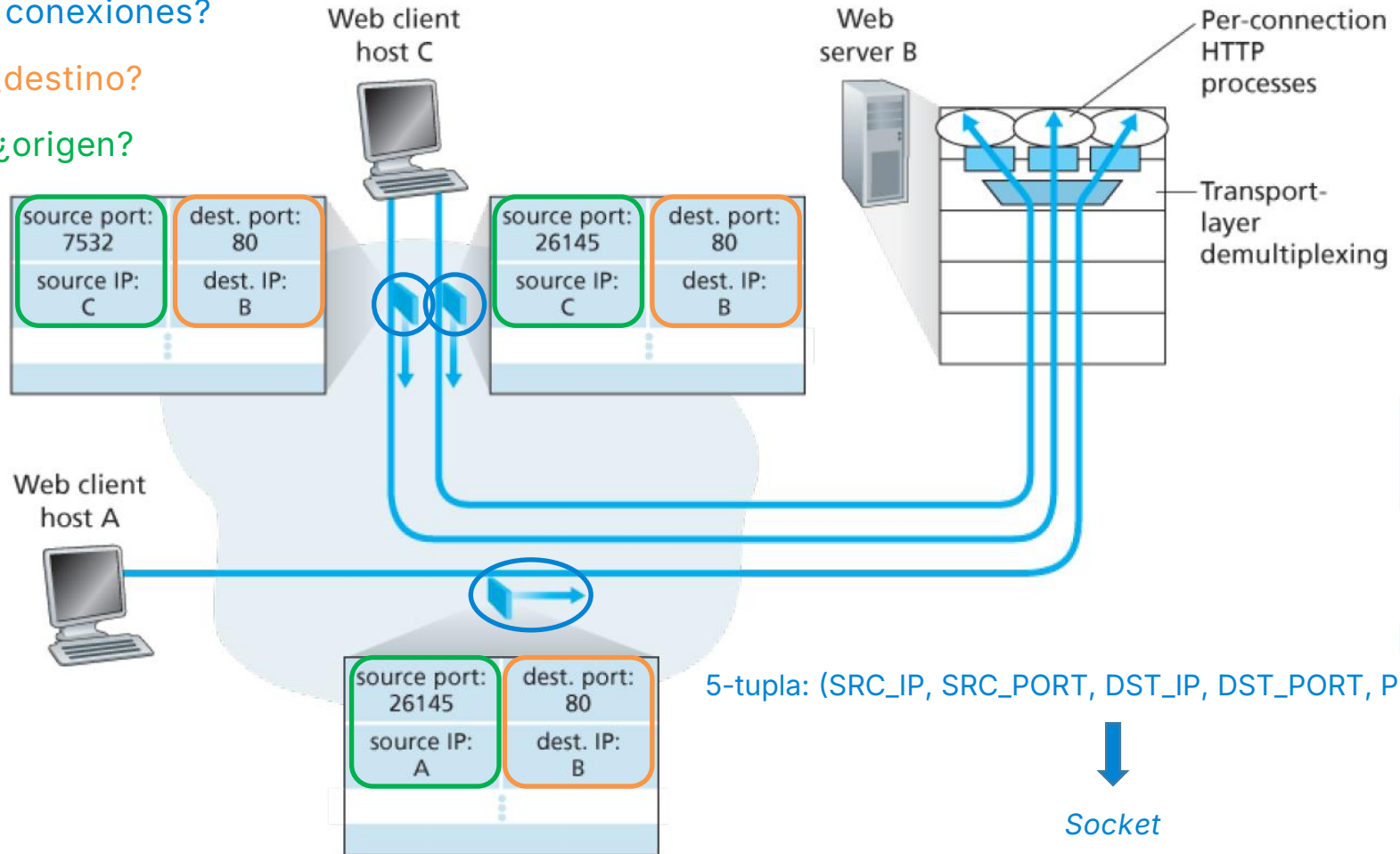
- TCP (RFC 793, 1981 y RFC 9293, 2022) es el protocolo de transporte emblemático de Internet y, junto con IP, da nombre a la arquitectura TCP/IP.
- **Ofrece un servicio ...**
 - ... **fiable**: porque:
 - garantiza la entrega sin errores, sin duplicados y ordenada empleando acuses de recibo, temporizadores, posibles retransmisiones y un *checksum* de cabecera y datos.
 - ... **orientado a conexión**: requiere establecer una conexión, mantenerla durante el intercambio y liberarla al finalizar.
 - ... **basado en flujos** (*streams*): por lo que
 - no delimita los mensajes y le corresponde a la aplicación delimitar e identificar sus mensajes si lo necesita.
- TCP incorpora:
 - **control de flujo** para no desbordar al receptor y,
 - **control de congestión** para adaptarse al estado de la red.

- TCP permite la multiplexación de diferentes aplicaciones simultáneamente empleando números de puerto.
 - Una aplicación puede usar varios puertos y mantener múltiples conexiones a la vez.
- Una conexión concreta se identifica por la 5-tupla: IP de origen, puerto de origen, IP de destino, puerto de destino, TCP), que se materializa en el sistema operativo como un **socket** (análogo a un descriptor de fichero).

¿# de conexiones?

¿destino?

¿origen?



- La **interfaz** habitual de TCP son los **sockets** (extremo local de una conexión identificado por una dirección IP y un puerto). Estos ofrecen un conjunto de **primitivas** al servicio del programador que se ofrecen como funciones de una biblioteca.
- Servidor**: entidad que expone un puerto (habitualmente) “bien conocido” ([*well-known ports*](#)) y espera conexiones.
- Cliente**: entidad que inicia la conexión empleando un puerto efímero.

Primitiva	Significado
SOCKET	Crea un nuevo punto terminal de comunicación
BIND	Asocia una dirección local con un socket
LISTEN	Anuncia la disposición de aceptar conexiones
ACCEPT	Establece en forma pasiva una conexión entrante
CONNECT	Intenta establecer activamente una conexión
SEND	Envía datos a través de la conexión
RECEIVE	Recibe datos de la conexión
SHUTDOWN	Desactiva selectivamente el envío o recepción de segmentos, pero no libera el <i>socket</i>
CLOSE	Libera la conexión

SRV & CLI: crean un SOCKET para identificar la conexión

SRV: BIND para asociar un puerto a la conexión

SRV: LISTEN para indicar al SO que debe escuchar

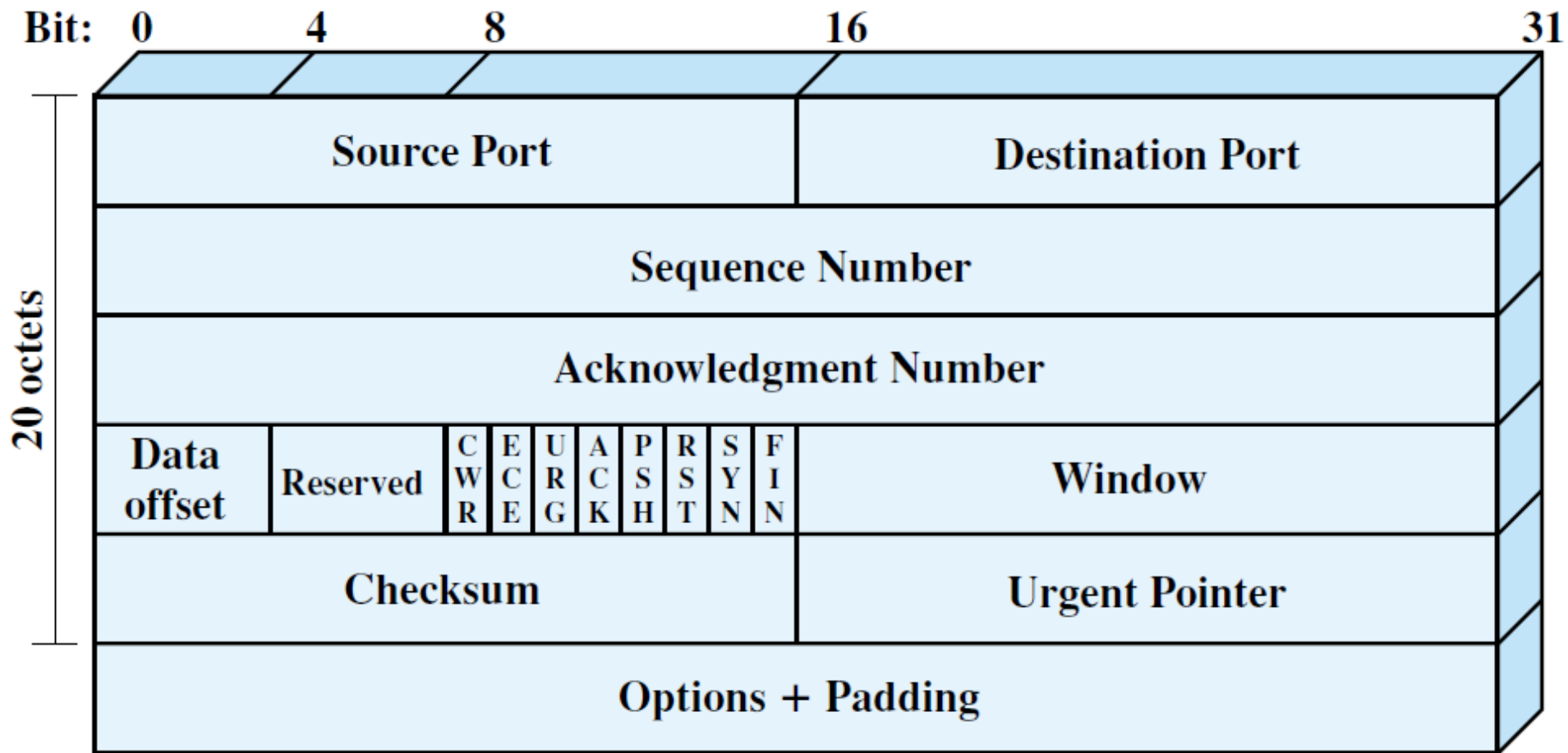
CLI: CONNECT para establecer la conexión al servidor

SRV: ACCEPT para aceptar la conexión entrante

SRV | CLI: SEND & RECEIVE para transmitir y recibir datos

CLI | SRV: SHUTDOWN para cerrar parcialmente la conexión

CLI & SRV: CLOSE para liberar la conexión



Tiene un tamaño de 20 bytes y opcionalmente, puede contener 40 bytes adicionales.

- **Source/Destination port** (16b): puertos empleados
 - 16 bits $\rightarrow 2^{16} = 65.536$ puertos ([listado completo en IANA](#))
 - El puerto 0 no se emplea
 - Puertos 1-1.023: asignados por la IANA para aplicaciones conocidas (“*well-known ports*”). Solo el administrador de un equipo (*root*) puede emplearlos. Ejemplos:
 - 20 y 21: FTP (datos en el 20 y control en el 21)
 - 22: SSH
 - 23: Telnet
 - 67 y 68: DHCP (servidor en el 67 y cliente en el 68)
 - 80: HTTP
 - 443: HTTPS
 - Puertos 1.024-49.151: puertos “registrados” por la IANA que se emplean en aplicaciones conocidas, pero se pueden usar sin privilegios de *root*.
 - Puertos 49.152-65.535: dinámicos, se emplean libremente.

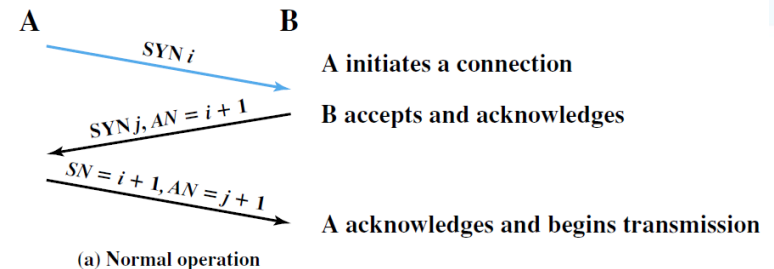
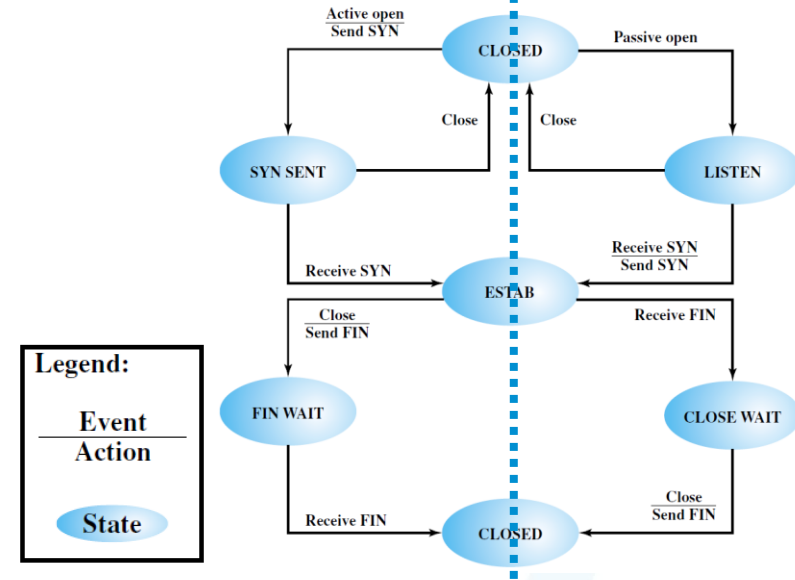
- **Sequence number** (32b): número de orden del primer byte de datos enviado en el segmento
 - Excepto si SYN==1 que entonces establece el valor de comienzo
- **Acknowledgement number** (32b): número del siguiente byte que se espera recibir cuando ACK==1.
- **Data offset** (4b): tamaño de la cabecera TCP, en palabras de 32 bits (4B), ≥ 5 (20 bytes).
- **Flags** (9b): indican situaciones de control:
 - NS: obsoleto (RFC 8311).
 - CWR: reducción de la ventana de congestión, acusa la recepción de un ECE.
 - ECE: notificación explícita de congestión a lo largo de la ruta o capacidad de ECN.
 - URG: especifica que el campo puntero urgente es válido.
 - **ACK**: indica que el campo número de confirmación es válido.
 - **PSH**: solicita entrega inmediata al proceso receptor sin esperar más acumulación.
 - **RST**: reinicia la conexión.
 - **SYN**: sincroniza los números de secuencia (*3-way handshake*).
 - **FIN**: finaliza el flujo emisor, ambos lados deben usarlo.

- **Window** (16b): indica cuántos bytes anuncia el receptor que puede aceptar (protocolo de control de flujo).
- **Checksum** (16b): suma de comprobación para detectar errores. Antepone una “pseudo-cabecera” UP (IP origen y destino, protocolo y longitud de TCP) de 12 bytes para evitar entregar un segmento TCP correcto que, por error, ha llegado al *host* equivocado → esto rompe la abstracción entre capas.
- **Urgent pointer** (16b): indica hasta dónde llegan los datos urgentes dentro del segmento si URG==1. Se recomienda no usarlo (RFC 6093, 2011).
- **Options** (variable): modificadores del comportamiento de TCP como:
 - **Window scale**: aumenta el tamaño de la ventana mediante el uso de un multiplicador de hasta 14 bits → máximo tamaño de ventana de ≈ 1 GiB.
 - **Maximum Segment Size (MSS)**: permite que el host especifique el tamaño máximo de segmento que puede aceptar (indica bytes de datos).
 - **Timestamp**: permite añadir marcas de tiempo para medir latencias.
 - **Selective Acknowledgement (SACK)**: permite la confirmación de bloques de datos concretos en presencia de pérdidas.
- **Padding** (variable): relleno de ceros hasta alinear a un múltiplo de 32 bits.

- TCP trabaja con un **flujo continuo de bytes** que lo divide en unidades de envío llamadas **segmentos**.
 - Esta segmentación es independiente de la aplicación: varias escrituras pueden combinarse en un solo segmento o una única escritura dividirse en varios.
 - Para recomponer el orden original se emplea el **número de secuencia**.
- La **carga útil** de cada segmento está acotada por el *Maximum Segment Size* (**MSS**). Este se anuncia durante el establecimiento de la conexión en los SYN y se elige en función de la MTU del camino para evitar fragmentación IP.
- **El valor del MSS influye en el control de flujo y congestión**, porque la ventana se mide en bytes y muchos algoritmos trabajan en unidades de MSS.
 - Para mejorar la eficiencia cuando la APP genera unidades pequeñas:
 - Envío: el **algoritmo de Nagle** combina envíos en un segmento más grande.
 - Recepción: los **ACK retardados** agrupan confirmaciones para disminuir la sobrecarga.

- **TCP contempla tres fases** básicas de la comunicación:
 - **Establecimiento** de la conexión
 - **Mantenimiento** de la conexión (transferencia de datos)
 - **Cierre** de la conexión
- La figura presenta un diagrama simplificado de los estados de TCP
 - Cada extremo tiene su propio estado
 - Un evento (local o remoto) causa una transición
- Izquierda: apertura/cierre activo
- Derecha: apertura/cierre pasivo
- **3-way handshake:**
 1. Solicitud de conexión con SYN activo y número de secuencia aleatorio i , llamado número de secuencia inicial (ISN).
 2. Confirmación de conexión ($AN=i+1$) y solicitud de apertura en el otro sentido con número de secuencia aleatorio (ISN) j .
 3. Confirmación de conexión ($AN=j+1$).
- También podría haber un establecimiento simultáneo, pero es muy infrecuente.

Fuente: Stallings (cap. 20)



- Durante la fase de mantenimiento, cliente y servidor pueden transmitir datos libremente siempre que el contrario tenga espacio para recibirlo.
- Según el estándar, una vez establecida la conexión, TCP por defecto no envía mensajes para mantener la conexión y si no hay tráfico la conexión no se cierra.
 - **Uno de los extremos se cae:** conexión semi-abierta
 - No es un problema grave ya que el nivel de aplicación se encargará de cerrar la conexión tras un periodo de inactividad o cuando el cliente no responda.
 - **Si se emplea NAT/PAT, el mapeo puede expirar tras un *timeout***
 - Se emplean sondas *Keep Alive* para solicitar periódicamente una petición de respuesta, son opcionales en TCP, pero la mayoría de las implementaciones actuales lo activan por defecto.

- Ambos extremos deben notificar el cierre con un mensaje FIN (numerado) y un ACK confirmando dicho FIN, en cada sentido.
 - **Se suele resumir en: FIN, FIN-ACK, ACK.**
- **Cualquiera de los dos extremos puede iniciar el cierre:**
 - Cierre activo/pasivo, según el caso, aunque normalmente es el cliente quien inicia la petición de cierre.
 - El cierre normal es ordenado y suele ser completo, aunque cierres parciales se permiten (un extremo no puede enviar datos, pero si recibirlos).
 - También puede darse un cierre simultaneo (ambos cierran activamente).
- **Tras el cierre, la misma conexión (IPs y puertos) no pueden reutilizarse durante un tiempo de $2 \times \text{MSL}$ ("Maximum Segment Lifetime", /proc/sys/net/ipv4/tcp_fin_timeout (2' en 1981, ahora 1').**
- **También existe un cierre abortivo** mediante el *flag* RST, que descarta el estado sin completar el intercambio de FIN e informa a la aplicación del error.
- **Los comandos netstat o ss muestran las conexiones del sistema.**

Necesito 2 voluntarios y el resto de la clase atenta a la presentación ...

Condiciones:

- el cliente envía segmentos de 10 B
- el servidor envía segmentos de 30 B
- asumir que tenemos una ventana infinita en ambos extremos

El cliente inicia el saludo con ...

SYN = 1, ACK = 0, FIN = 0

Seq. #: 77

Ack. #: --

TCP Segment length: 0

El servidor sincroniza y acusa
el recibo de la solicitud de
conexión con ...

SYN = 1, ACK = 1, FIN = 0

Seq. #: 14

Ack. #: 78

TCP Segment length: 0

El cliente acusa el número de
secuencia inicial del servidor y
comunicación establecida ...

SYN = 0, ACK = 1, FIN = 0

Seq. #: 78

Ack. #: 15

TCP Segment length: 0

Y entramos en la fase de mantenimiento de la conexión ...

El cliente envía datos y vuestro compañero tiene estas 2 tarjetas posibles, ¿cuál usará?

SYN = 0, ACK = 0, FIN = 0

Seq. #: 78

Ack. #: 0

TCP Segment length: 10

SYN = 0, **ACK = 1, FIN = 0**

Seq. #: 78

Ack. #: 15

TCP Segment length: 10

Cualquiera de las dos, el número de secuencia es correcto y la diferencia es que una aprovecha para volver a confirmar la trama que recibió previamente, por si acaso ...

La APP del servidor no responde inmediatamente con datos y la implementación de TCP responde con un ACK solamente ...

SYN = 0, ACK = 1, FIN = 0

Seq. #: 15

Ack. #: 88

TCP Segment length: 0

Posteriormente, decide enviar 30 B de datos... y encima lo hace solicitando su entrega inmediata ... y al igual que antes, aprovecha para confirmar, aunque el segmento previo ya lo hizo...

ACK = 1, PSH = 1

Seq. #: 15

Ack. #: 88

TCP Segment length: 30

Cualquiera de los extremos puede enviar múltiples mensajes siempre que haya espacio y estamos en una situación de ventana infinita ...

El cliente vuelve a enviar 10 B y aprovecha para confirmar el segmento que envió el servidor

SYN = 0, ACK = 1, FIN = 0

Seq. #: 88

Ack. #: 45

TCP Segment length: 10

El servidor confirma los datos recibidos y no tiene datos que enviar.

SYN = 0, ACK = 1, FIN = 0

Seq. #: 45

Ack. #: 98

TCP Segment length: 0

El cliente
envía dos
segmentos
seguidos
con la mala
suerte que
el primero
se pierde y
no llega al
servidor ...

SYN = 0, ACK = 0, FIN = 0

Seq. #: 98

Ack. #: --

TCP Segment length: 10



SYN = 0, ACK = 0, FIN = 0

Seq. #: 108

Ack. #: --

TCP Segment length: 10



El servidor no reconoce el # seq. recibido, descarta la trama y envía el último ACK tal como hizo antes.

SYN = 0, ACK = 1, FIN = 0

Seq. #: 45

Ack. #: 98

TCP Segment length: 0

Luego si no estoy usando rechazo, ni me estoy guardando el segmento que ha llegado fuera de orden, ¿qué mecanismo estoy implementando?

Go-back N

El cliente vuelve a reenviar los dos segmentos desde el último que le hace ACK el servidor ...

Tras esto... el servidor
confirma su recepción
(acumulativa) y envía los
datos que tenias pendientes...

SYN = 0, ACK = 1, FIN = 0

Seq. #: 45

Ack. #: 118

TCP Segment length: 30

Y pasamos a cerrar la conexión... pero no olvidemos que el servidor nos acaba de enviar 30 B de datos que no hemos confirmado.

SYN = 0, **ACK** = 1, FIN = 0

Seq. #: 118

Ack. #: 75

TCP Segment length: 0

SYN = 0, ACK = 0, **FIN** = 1

Seq. #: 118

Ack. #: --

TCP Segment length: 0

FIN

o ...

SYN = 0, **ACK** = 1, **FIN** = 1

Seq. #: 118

Ack. #: 75

SYN = 0, **ACK** = 1, **FIN** = 1

Seq. #: 75

Ack. #: 119

FIN+ACK

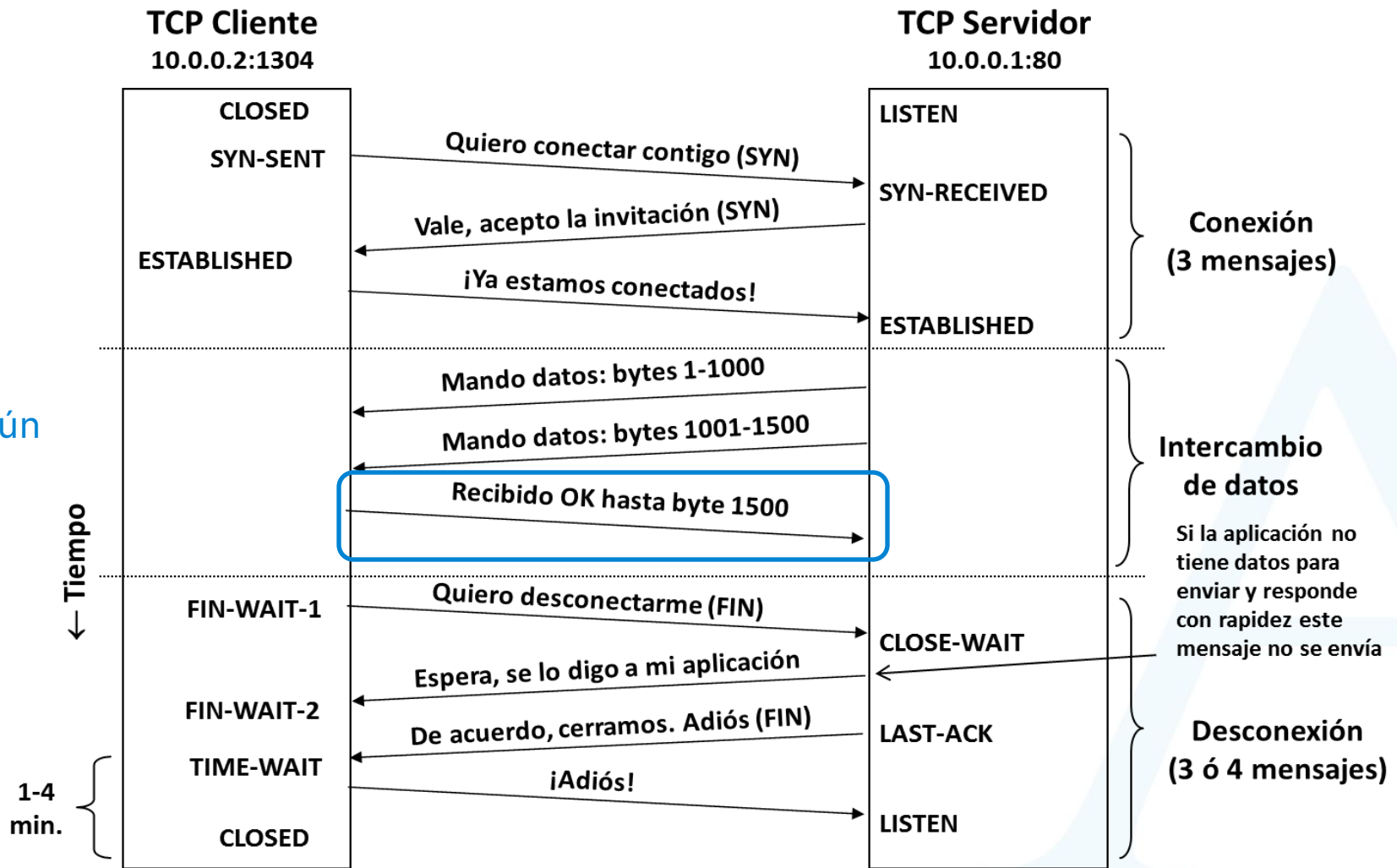
SYN = 0, **ACK** = 1, FIN = 0

Seq. #: 119

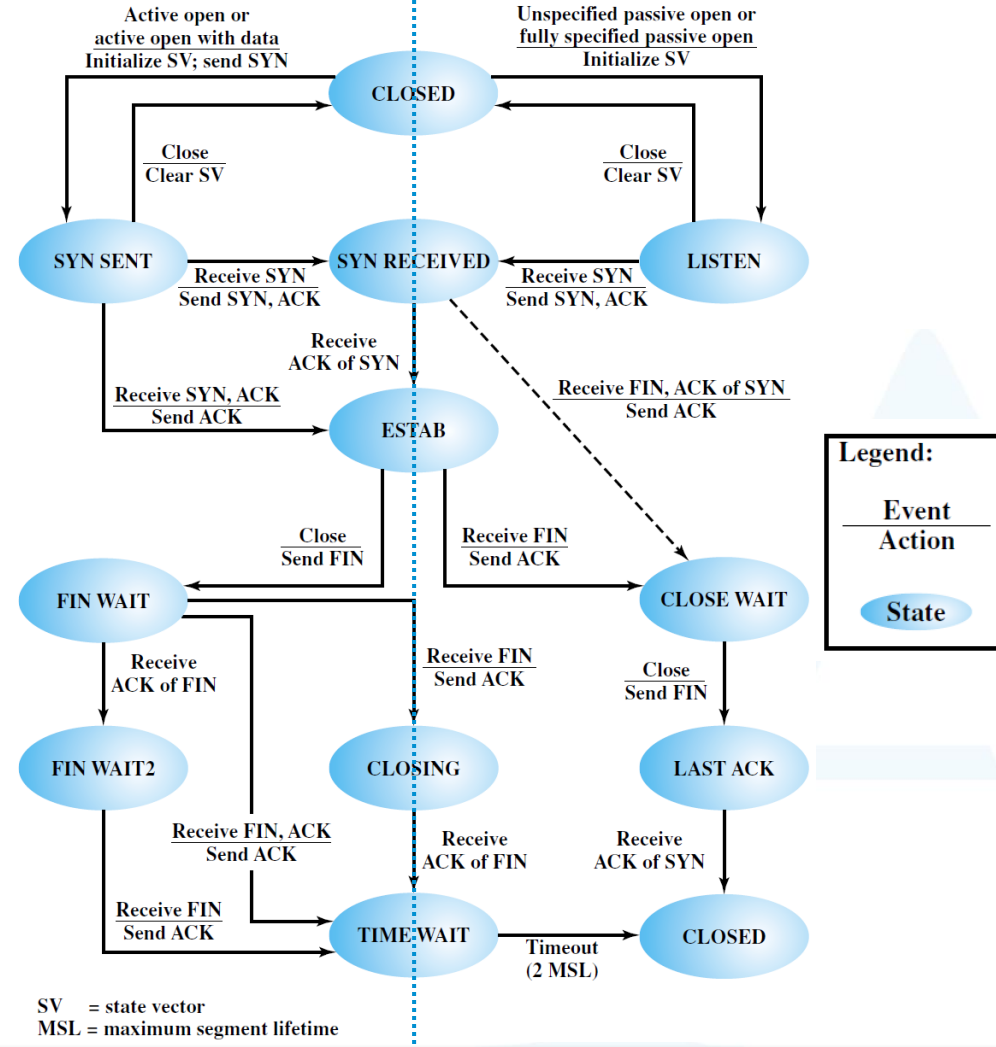
Ack. #: 76

ACK

¿Veis algún error?



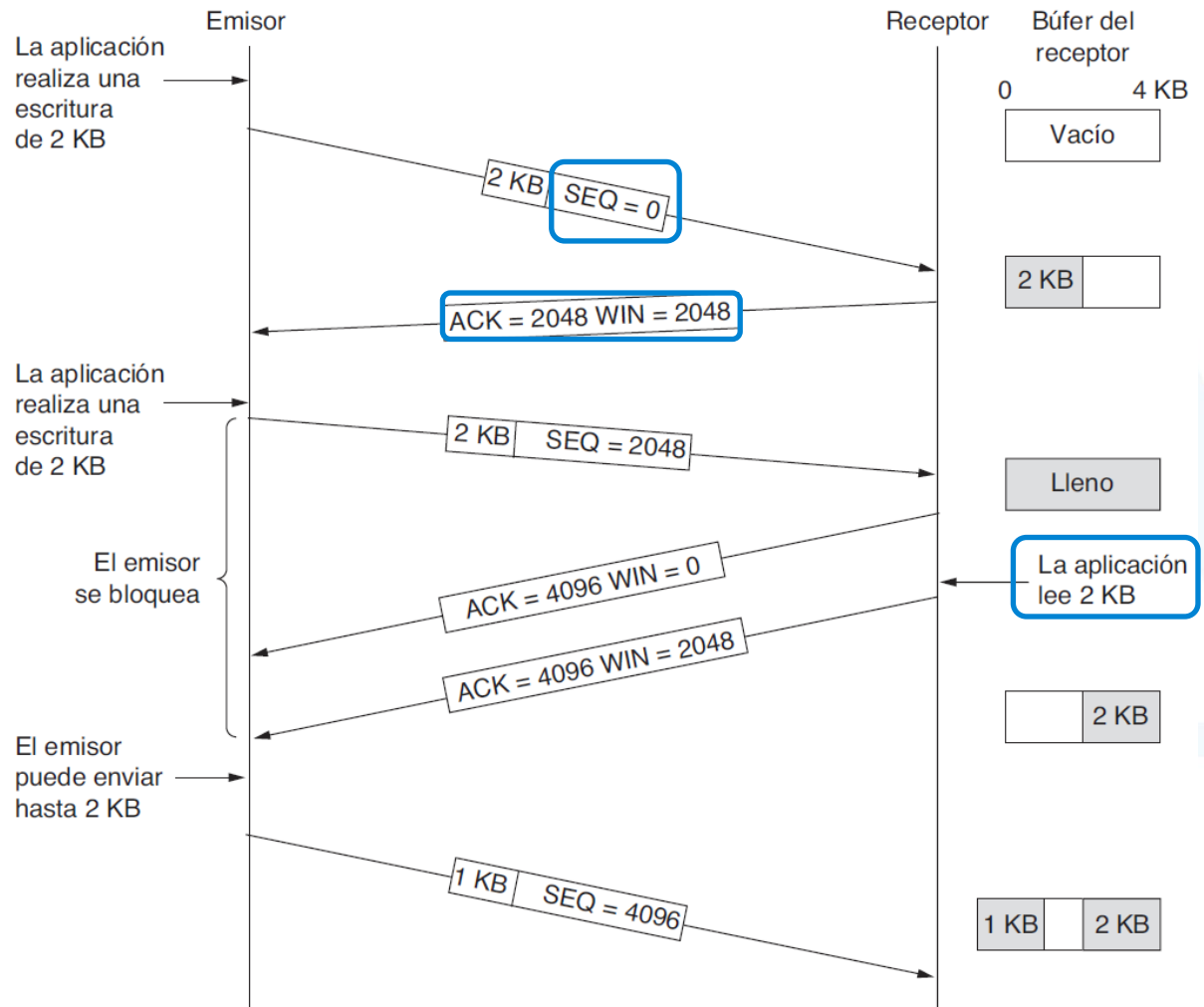
- La figura muestra el diagrama de estados completo de TCP.
- Izquierda: apertura/cierre activo.
- Derecha: apertura/cierre pasivo.



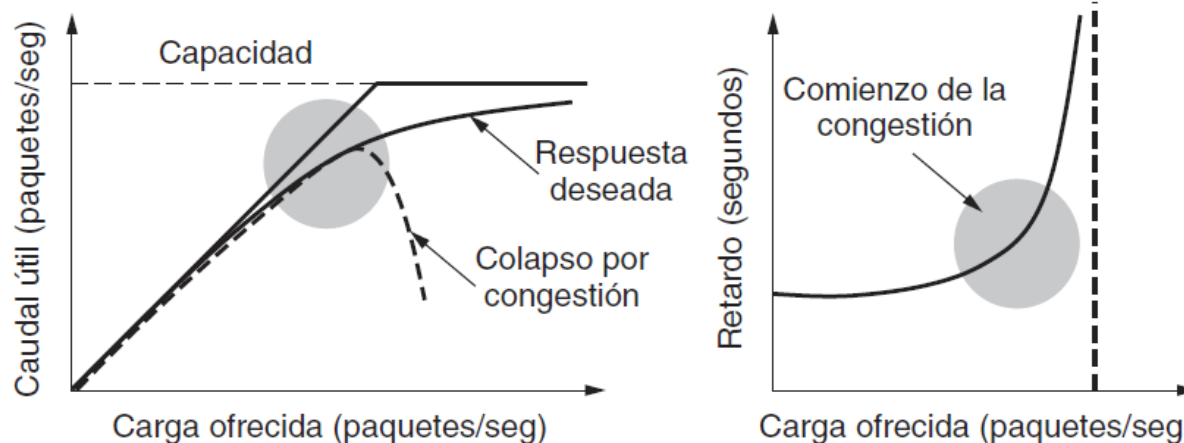
- **TCP implementa**, por defecto, ARQ con vuelta atrás N como mecanismo de control de flujo y errores, es decir, se basa en **un control de flujo mediante ventana deslizante**.
- Cada extremo anuncia cuántos bytes puede aceptar → **ventana de recepción**.
 - Campo *Window* de 16 bits → 64 KB como máximo salvo que se emplee la opción de *Window scale*.
 - El emisor puede tener “en vuelo”, como máximo, esa cantidad de información y avanza la ventana a medida que recibe confirmaciones.
 - El receptor, si se satura, puede anunciar una **ventana de valor 0**.
 - Para no quedar bloqueado en ese estado, el emisor puede enviar sondas de ventana al receptor a intervalos crecientes para detectar cuando este vuelve a tener espacio y puede reanudar el envío.
- La **confirmación es acumulativa** y el campo *ACK Number* indica el siguiente **byte** que se espera recibir de la secuencia.
 - Si se han recibido 5 segmentos de 1000 bytes consecutivos (numerados del 0 al 4999), se puede confirmar el último con el campo *ACK Number* igual a 5000.
- La detección de errores se realiza mediante el *checksum*.
- La retransmisión se basa en temporizadores y en confirmaciones.
- Además de por tiempo de espera, el emisor puede detectar pérdidas mediante la recepción de confirmaciones duplicadas que señalan el mismo hueco → **retransmisión rápida** del segmento faltante sin esperar al temporizador de retransmisión.
- Si ambos extremos permiten las confirmaciones selectivas, el receptor informa de bloques recibidos fuera de orden, lo que reduce las retransmisiones.

Ejemplo con una ventana de recepción de 4 KB.

- No es necesario que la numeración comience en 0.
- La numeración es mayor que la ventana de recepción, tal como establece ARQ con vuelta atrás N.
- Cuando la aplicación lee, no puede saber si le han enviado 2 mensajes de 2 KB, 4 de 1 KB o 1 de 4 KB.
- SEQ indica el índice del primer byte que se está transmitiendo, ACK indica el del siguiente que se espera recibir y WIN la ventana disponible.



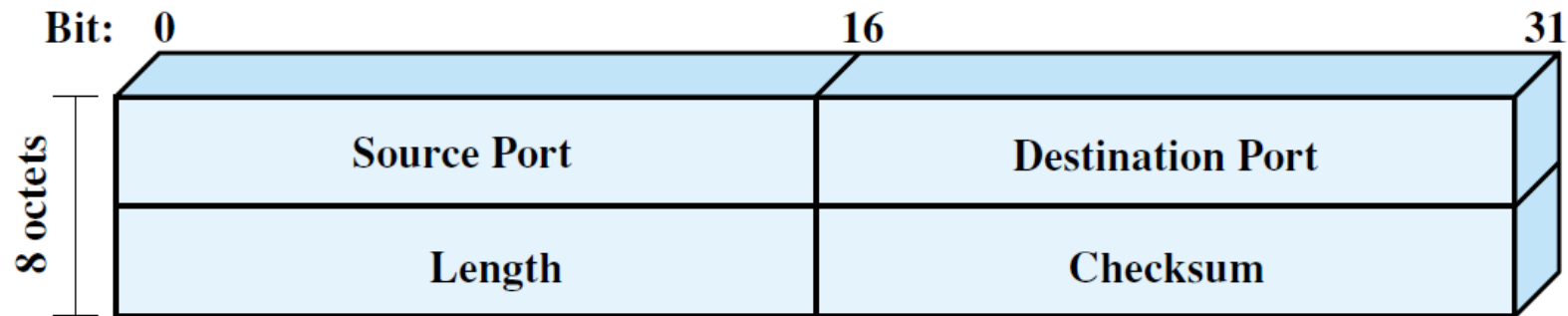
- La **congestión** en una red es la situación de pérdida de rendimiento (*throughput*, latencia) causada por un **exceso de tráfico ofrecido a la red**, que provoca pérdidas de datos, retransmisiones, ...
- Efectos típicos:
 - Congestión leve: aumento de las latencias (*round-trip time*, RTT).
 - Congestión grave: pérdida de paquetes.



- **Para evitar la congestión debe reducirse la tasa de inyección.**
- Se emplea un mecanismo de ventana con tamaño variable, análoga a la empleada para control de flujo: “**ventana de congestión**”, controlada por el transmisor.
 - Pequeña → se evita, pero la velocidad promedio de envío es baja.
 - Grande → la velocidad promedio de envío aumenta a costa de un mayor riesgo de congestión.
 - La ventana será: $\min(\text{ventana de congestión}, \text{ventana de recepción})$.
- TCP interpreta las pérdidas como señal de congestión y ajusta la ventana de congestión de forma controlada. Además, si ECN está habilitado, puede reaccionar en base a las ECNs recibidas, sin necesidad de que haya pérdidas.
- TCP (originalmente) define dos mecanismos diferentes de gestión:
 - **Arranque lento** (*slow start*): típicamente solo al comienzo
 - La ventana de congestión parte de un valor pequeño (1 o unos pocos MSS) y se duplica cada RTT acumulando confirmaciones hasta alcanzar un **umbral de congestión**, a partir del cual:
 - **Evitación de la congestión**: el aumento de la ventana es más conservador (1 MSS x RTT).

- UDP ([RFC 768](#), 1980) es un protocolo de transporte no orientado a conexión, sin control de flujo ni errores → no fiable
 - UDP presta un servicio ligero y muy eficiente que aporta lo imprescindible: multiplexación de aplicaciones y direccionamiento.
- Escenarios típicos de aplicación:
 - Recolección de datos: cuando se reciben datos periódicamente (ej.: sensórica) y no hay problemas por perder alguno.
 - Diseminación de información: UDP permite el envío *broadcast* y *multicast*, TCP no (¿quién confirma?)
 - Petición-respuesta: aplicaciones sencillas con un patrón de petición-respuesta (ej.: DNS) donde la aplicación gestiona las retransmisiones.
 - Aplicaciones en tiempo real: como voz, vídeo, telemetría, donde se prioriza la baja latencia frente a la recuperación de pérdidas.
- Aunque la PDU de transporte es el segmento, en UDP se habla de **datagrama**.

- **Puerto de origen/destino** (16b): número de puertos empleados.
- **Longitud** (16b): longitud total (cabecera + datos) en bytes.
- **Checksum** (16b): suma de comprobación para detectar errores tanto en los datos como en la cabecera.
 - Si un paquete está corrupto → ¡se descarta!



Redes de Ordenadores

GII – IYA022

Mariano Benito